上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/9

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

软件的生命周期

模块 5 · 重新认识软件（1/4）

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件的生命周期

本讲位置：从「结构」到「时间轴」

上一讲 · L-04

架构案例分析：电商系统从单机到云原生的 11 个阶段

由组件构成系统：每一步都遵循 瓶颈 → 方案 → 新问题

关注软件的「结构」：由哪些部分组成、如何连接

本讲问题

一行代码从编写到退役，要经历哪些状态？

每个状态的产出物是什么？能否回到上一个状态？

引例

选课平台今天 9:00 开放选课。8:55 发现刚上线的新功能存在缺陷，此时可以采取哪些措施？

在课程实践中解答

本讲结构

1 为什么要重新认识软件

2 制品的生命周期

3 实例的生命周期

4 职责归属

5 课程实践：选课平台的一次发布

本模块 重新认识软件（L-05 ~ L-08）：有生命周期的制品（本讲）· 运行中的进程（L-06）· 资源的消费者（L-07）· 可度量的对象（L-08）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、为什么要重新认识软件

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件的生命周期

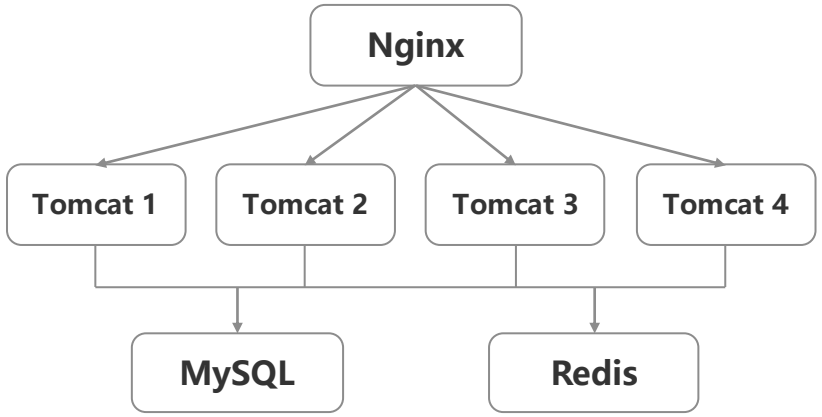
为什么重新认识

架构图中的软件与时间轴上的软件

架构图中的软件 = 结构

关注：由哪些部分组成、如何连接

开发视角通常关注此图



此图回答「有哪些组件、如何连接」

时间轴上的软件 = 一系列状态

关注：当前处于什么状态、如何进入下一状态、能否回退

运维视角通常关注此图



每天可能发生多次；每个版本都将被下一个版本替换

要点 DevOps 要求同时关注两张图；L-01 所述的 SaaS、持续、长生命周期、迭代等特点均体现在右图中

来源：L-01、L-04 讲义

为什么重新认识

两条时间轴：制品的生命周期与实例的生命周期

制品的生命周期 · 跨越多个版本 · 数天至数年

源码 → 构建 → 测试 → 版本 → 部署 → 运行 → 变更 → 下线

实例的生命周期 · 一次运行之内 · 数分钟至数周（在「运行」阶段内多次重复）

启动 → 就绪 → 服务 → 停止 → 替换

…替换之后，新实例从「启动」重新开始

制品轴回答

该版本是什么、部署在哪些环境、能否回退

实例轴回答

该进程当前能否接收流量、如何安全停止

要点 两条轴的交点是「部署」：将某个制品实例化为若干运行实例。后续各页均归属于其中一条轴

量化视角：DORA 四项指标

指标	衡量内容	精英团队
部署频率	向生产环境部署的频率	按需，每天多次
变更前置时间	从提交到上线生产所需时间	小于一天
变更失败率	部署后需要修复或回滚的比例	约 5%
失败部署恢复时间	部署失败后恢复服务所需时间	小于一小时

如何解读这四项指标

前两项衡量速度，后两项衡量稳定性；DORA 的研究发现二者并不矛盾，表现优秀的团队在两方面均表现良好

要点 这四项指标反映生命周期本身的健康状况，L-08 讲授指标时再次讨论

来源：DORA, Accelerate State of DevOps Report（dora.dev）；数值以最新年度报告为准

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、制品的生命周期

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

制品 · 总览

制品生命周期总览：九个节点及其产出物



产出物是什么?

没有产出物的环节不构成独立阶段；产出物决定了该阶段能否被追溯与复现

常见问题是什么?

每个节点按「问题 → 原因 → 对策」分析

能否回到上一个状态?

不可回退的变更须格外谨慎，应安排在最后并留足观察时间

说明 每个箭头代表一次状态变更；本节对每个节点依次提出这三个问题

源码与提交

产出物：一次提交

生命周期的最小变更单位

其后的制品、版本与部署都必须能追溯到该提交

两道门禁

代码评审：人工检查

CI：自动检查，构建与测试通过后方可合入主干

对应 L-02 CI 原则中的「每日提交」与「自检构建」

原则

分支生命周期短，小步提交

主干始终处于可发布状态

分支模型（主干开发与长期分支、分支保护）的具体做法见实验 1 指导书

要点 提交是追溯的起点：「线上运行的是哪次提交」由此可以回答

来源：L-02 Part A（CI 原则）

构建与制品：不可变制品

问题	原因	对策
「在我的机器上可以运行」	构建与运行环境不一致：依赖版本、系统库、JDK 小版本、环境变量	一次构建、多处部署（build once, deploy many）；制品不可变，环境差异只允许来自配置
同一份源码两次构建结果不同	依赖未锁定版本（使用「最新版」）；基础镜像使用 latest 标签	使用锁文件（package-lock / go.mod 与 go.sum / Maven 固定版本）；镜像按 digest 引用；构建环境本身也以镜像固定
制品库膨胀，无法确定线上运行的是哪个制品	缺少保留策略；制品未与提交建立对应关系	制品标注 git SHA 与构建号；按规则保留：最近 N 个及所有已上线的制品

要点 制品是生命周期中的「不动点」：此后不再修改，只与配置组合并部署到环境中；容器镜像将程序、依赖与运行环境一并固定（参见 L-02）

来源：The Twelve-Factor App, II. Dependencies；V. Build, release, run

测试：在生命周期中的位置

构建时	test 环境	staging	线上
单元测试 · 秒级 · 最多	集成测试 · 分钟级	端到端、性能 · 小时级	合成监控、金丝雀 · 最少

问题	原因	对策
发布前集中测试两天，发现问题后返工并再测试两天	反馈周期过长	测试左移：成本越低的测试越早执行；门禁分层：构建门禁、合并门禁、发布门禁
测试环境验证通过，上线后仍出现故障	环境漂移	同一制品在各环境中运行，并以代码定义环境（见第 15 页）

要点 测试不是一个独立阶段，而是贯穿每次状态变更的门禁

来源：L-02 Part A（测试自动化）；M. Fowler, TestPyramid

版本：为制品命名

MAJOR	MINOR	PATCH
不兼容的改动	向后兼容的新功能	向后兼容的修复

SemVer：主版本.次版本.修订号，如 1.4.2

标识	使用者	回答的问题
版本号 1.4.2	依赖方、用户	兼容性承诺
构建号 #1873	流水线	哪一次构建
git SHA a1b2c3d	开发者	哪一份源码

三者须能相互查询：制品同时标注三类标识；「线上运行的是什么」必须能够直接回答

要点 SemVer 的意义在于承诺而非数字：MAJOR 变化意味着依赖方需要修改代码；Tomcat 10 由 javax 迁移到 jakarta 即为一次 MAJOR 变更（L-04）

来源：semver.org；Apache Tomcat Migration Guide

三个动词：发布 / 部署 / 放量

动作	改变的状态层	示例	撤销方式
发布 release	制品层：制品获得版本号，可供部署	打标签 v1.4.2，推送至制品库	撤回版本（较少使用）
部署 deploy	环境层：制品被放进某个环境并运行	v1.4.2 运行在 prod 的 4 台机器上	重新部署上一版本
放量 rollout	流量层：使用新版本的用户比例	1% 用户使用新版本；打开功能开关	切回流量 / 关闭开关

L-02 中 Continuous Delivery（随时可发布）与 Continuous Deployment（自动部署）的区别对应前两层

将「部署」与「放量」分离是互联网产品最重要的生命周期实践之一：代码可以先部署到生产环境而不对用户可见（dark launch）

要点 撤销速度自上而下递增：切回流量以秒计，重新部署以分钟计；课程实践将用到这一点

注：英文 release 既指「发布版本」也指「向用户开放」，本课按上表区分。来源：Humble & Farley (2010)；Hodgson, Feature Toggles (2017)

环境与配置：同一制品，不同配置

dev	test	staging	prod
开发者的个人环境	运行自动化测试	尽量与生产一致，供预演	生产

问题	原因	对策
数据库地址、密钥硬编码在代码中，更换环境需修改代码并重新构建	配置未与代码分离	配置由环境注入（环境变量 / 配置中心）；密钥单独管理，不纳入代码仓库
修改一个配置项导致服务全部不可用	配置变更未按变更流程管理：缺少评审、灰度与回滚	配置同样纳入生命周期管理：版本化、评审、先灰度后全量、可回滚（Q-2 第五幕的「配置失误」）
staging 验证通过，但 prod 表现不同	环境由人工搭建，随时间推移发生漂移	以代码定义环境（IaC），通过重建而非修补来维护环境；模块 6、7 展开

要点 生命周期中真正可变的只有两项：制品版本与配置版本；二者都必须能回答「当前是哪一版、上一版是什么」

来源：The Twelve-Factor App, III. Config；X. Dev/prod parity

部署策略对比：解决的问题与增加的成本

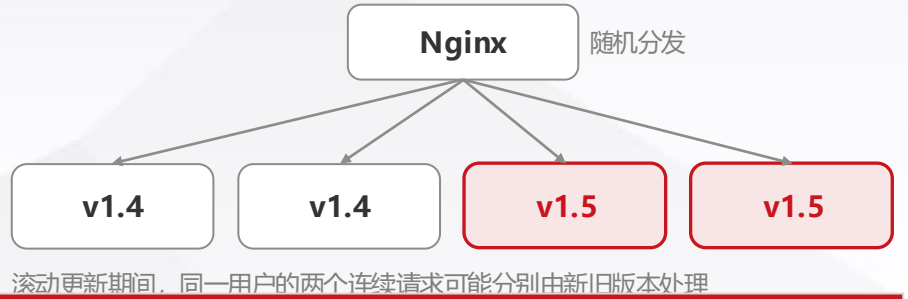
策略	做法	解决的问题	增加的成本	回退速度
重建（停机）	停止旧版本，启动新版本	实现最简单	存在停机窗口，用户可感知	分钟级：重新部署
滚动	每次替换一台（或一批）	无需停机	新旧版本共存，须向后兼容；替换期间容量下降	分钟级：反向滚动
蓝绿	两套环境，切换流量	切换与回退均为秒级	资源翻倍（至少在切换窗口内）；数据库仅一套，schema 仍须兼容	秒级：切回
金丝雀	先导入 1%～10% 流量观察指标，再逐步放量	以真实流量验证，影响范围可控	需要按比例分流的设施与指标对比能力；新旧版本共存时间更长	秒级：切回流量
功能开关	代码已部署，功能由配置决定是否生效	部署与放量完全分离；回退最快	代码中增加分支；开关须及时清理，否则形成技术债；开关本身属于配置变更，需纳入管理	秒级：关闭开关

要点 不存在普遍最优的策略，取决于回退速度要求与可接受的成本；实践中常组合使用滚动、金丝雀与开关（参见 L-02 部署模式、L-03 灰度与回滚）

来源：M. Fowler, BlueGreenDeployment (2010)；D. Sato, CanaryRelease (2014)

制品 • 运行

版本共存与向后兼容



问题	原因	对策
第一个请求由 v1.5 处理并返回新格式数据，第二个请求由 v1.4 处理时解析失败	API / 数据格式不向后兼容	只增加字段，不删除字段、不改变语义；删除字段前先标记废弃，若干版本后再删除；消费者容忍未知字段（tolerant reader）
v1.5 写入队列的消息，v1.4 的消费者无法解析	消息格式发生变化	消息携带版本号；先升级消费者、后升级生产者（先能读取新格式，再写入新格式）
v1.5 修改了 session / 缓存格式，v1.4 读取时异常	共享状态的格式发生变化	更换 key 前缀，或实现兼容读取

要点 只要不停机，新旧版本共存即为常态；向后兼容不是可选项，而是必要条件；schema 的兼容见后续两页

来源：M. Fowler, TolerantReader (2011)

回滚的边界

可回滚：代码与配置层

制品：重新部署上一版本

配置：切回上一版配置

流量：切回旧版本 / 关闭开关

无法自动回滚：状态

schema：已删除的列无法恢复；新增 NOT NULL 约束会使旧代码写入失败

已写入的数据：新逻辑写入的记录，旧代码无法识别

已发出的消息：短信一经发送即无法撤回

缓存：新格式的缓存项仍然存在，旧代码读取时出错

外部副作用：已调用第三方接口、已完成扣款

回滚的真正含义

使系统回到一个已知正确的状态；状态 = 代码 + 配置 + 数据

两种修复方式

回滚（roll back）与向前修复（fix forward）；涉及数据修改的变更通常只能向前修复

为回滚而设计

将代码变更与数据变更拆分为不同的发布；写入数据前先评估「旧代码读取该记录时会如何处理」

要点 代码与配置可以按版本回退，数据只能向前演进

来源：Google SRE Book, Ch. 8 Release Engineering；Humble & Farley, Continuous Delivery, Ch. 12 Managing Data

schema 变更：expand–contract

- 场景：enrollment 表的冲突判定由「按课程」改为「按教学班」，需新增 section_id 列

① 扩展 expand

新增可空列 section_id

仅修改 schema，不修改代码

旧代码不受影响

回滚：无需回滚，schema 向后兼容

→

② 双写 migrate

新代码同时写入两列，后台回填历史数据

读取仍使用旧列

回滚：换回旧代码，保留新列不影响运行

→

③ 切读

新代码改为读取 section_id

此时数据已完整

回滚：换回上一版本，读取旧列；旧列仍在双写

→

④ 收缩 contract

确认旧列已无读取方

再删除旧列 / 添加 NOT NULL

不可回滚：因此放在最后，并留足观察时间

时间轴向右：每步对应一次发布，均可单独回滚；仅最后一步不可回退

要点 每步都同时兼容前后两个版本，因此任一步骤均可退回上一步；代价是周期较长，且双写带来一致性成本。在一次发布中同时修改 schema 与代码，将导致该发布无法回滚

来源：D. Sato, ParallelChange (martinfowler.com, 2014)；Humble & Farley, Continuous Delivery, Ch. 12

下线与退役

问题	原因	对策
依赖的运行时 / 库已到 EOL，不再提供安全补丁	依赖同样有生命周期，且不受使用方控制	盘点依赖的 EOL 日期（endoflife.date），将升级纳入路线图。例如：Tomcat 8.5 移除 BIO、10.1 移除 APR（L-04）；CentOS 7 于 2024-06-30 EOL
旧接口无人敢删除，因为无法确认是否仍有调用方	缺少功能废弃流程	标记 deprecated → 监控调用量 → 通知调用方 → 设定下线日期 → 下线 → 删除代码
已下线功能遗留的数据、队列与定时任务仍在运行	只删除了代码，未清理相关资源	下线清单：代码、配置、数据（归档或删除，注意合规保存期）、基础设施、监控告警、文档

要点 缺少退役阶段的生命周期会持续累积债务；「长生命周期」也意味着软件将经历其所有依赖的生命周期

来源：endoflife.date；Apache Tomcat Migration Guide

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、实例的生命周期

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件的生命周期

实例

一次运行的状态机



启动中

读取配置、建立数据库 / Redis 连接池、加载缓存或模型、JIT 预热

进程已运行，但尚不能提供服务

已就绪

依赖均可用，可接收流量

服务中

正常服务

期间可能出现「存活但不健康」：线程池耗尽、依赖中断

停止中

收到停止信号 → 停止接收新请求 → 处理完在途请求 → 关闭连接 → 退出

超时则被强制终止

要点 制品轴上的一次「部署」，在实例轴上表现为「启动一批新实例、停止一批旧实例」；各种部署策略都是对这一状态机的编排

来源：The Twelve-Factor App, IX. Disposability

就绪与存活：健康检查的两个问题

存活：进程是否仍正常运行?
进程存在且能够响应；否则重启

就绪：能否接收流量?
依赖可用、预热完成、未过载；否则暂停分配流量，但不重启

问题	原因	对策
滚动更新时新实例启动后立即接入流量，最初几秒持续返回 500	只检查进程是否启动，未检查是否能够提供服务	就绪检查通过后再加入负载均衡（Nginx / LB 健康检查；参见 L-04 阶段③、2025 实验 3 的 /healthz）
实例僵死（死锁、OOM 后部分功能失效）但进程仍在，流量持续进入并超时	缺少存活检查，或仅检查进程是否存在	存活检查探测真实处理路径；连续失败 N 次后重启
数据库短暂抖动导致所有实例的存活检查同时失败，全部重启，引发雪崩	将「依赖不可用」误判为「自身故障」	依赖故障只影响就绪状态，不影响存活判断；存活检查不依赖外部服务

要点 两个问题的答案不同，处理方式也不同；Kubernetes 将其实现为 livenessProbe 与 readinessProbe（另有 startupProbe），模块 7 将进一步讨论

来源：Kubernetes docs, Liveness, Readiness and Startup Probes

优雅停止与启动时间

优雅停止的五个步骤及常见问题

1 收到 SIGTERM（而非 SIGKILL）：进程须注册该信号的处理

2 从负载均衡中摘除 / 将就绪状态置为失败：否则仍会收到新请求

3 等待在途请求完成：须设上限（如 30 s），否则单个慢请求会拖延整个发布

4 关闭连接池、刷新日志、退出

5 超时仍未退出 → 强制终止，在途请求丢失

代价

发布时间 = 实例数 ×（停止 + 启动 + 就绪）/ 并行度

长连接（WebSocket、SSE，L-03）需主动通知客户端重连，排空（drain）耗时更长

启动时间为何是硬约束

决定扩容响应速度：流量到达后，新实例可能需数分钟才能投入服务

决定回滚速度，以及能否采用 Serverless（冷启动）

L-03 中 Quarkus / GraalVM 的对比正是为了缩短启动时间，下一讲展开

要点 优雅停止与快速启动是实例能够被频繁替换的前提

来源：The Twelve-Factor App, IX；Kubernetes docs, Pod Lifecycle（默认终止宽限期 30 s）

实例生命周期短，要求服务无状态

实例随时可能被替换
部署、扩缩容、故障、机器回收

→

实例内的数据随时可能丢失
内存中的 session、本地文件、本地缓存

→

状态须存放在实例之外
Redis / 数据库 / 对象存储

→

实例才能随时终止、随时启动

→

进而支持滚动更新、扩缩容与自动恢复

回顾

L-04 阶段③引入 Nginx 后出现的「Session 共享」问题，根源不在负载均衡，而在于实例不再唯一、不再长期存在

L-02 云原生：无状态、水平扩展

推论

无状态不是架构偏好，而是实例生命周期缩短的必然结果

反之，有状态组件（数据库、队列）的实例生命周期须较长，替换须谨慎；这是模块 6 要区分的两类工作负载

要点 无状态是由实例生命周期短这一约束决定的设计，而非风格选择

来源：The Twelve-Factor App, VI. Processes

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、职责归属

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

职责归属

各阶段的责任方 · 流水线即生命周期的自动化

阶段	传统责任方	DevOps 下的责任方
源码 / 构建 / 测试	开发	开发
版本 / 发布	发布工程 / 测试	开发（自动化）
环境 / 配置 / 部署	运维	开发 + 平台（自助）
运行 / 监控 / 值班	运维	开发（you build it, you run it）+ SRE
下线	无明确责任方	产品 + 开发

流水线是这条生命周期的自动化

每个箭头对应一个流水线阶段，每道门禁对应一次自动检查；L-02 讲授如何搭建，本讲说明它管理的对象

要点 DevOps 不是一个岗位，而是将生命周期的所有权交还给编写代码的人，由平台团队提供自助工具（参见 L-02「DevOps 与架构」）

来源：L-02 讲义；Google SRE Book, Ch. 8

实验 1 预告：在 GitHub 上搭建从提交到制品的流水线

实验内容

代码仓库 + 分支保护（合并门禁）

分支模型的具体做法见指导书

GitHub Actions：push → 构建 → 测试 → 打包制品 / 容器镜像 → 标记版本 / release

产出：能够回答「线上运行的是哪次提交」的制品

对应本讲

源码与提交 → 构建与制品 → 测试 → 版本 → 发布

每一步都是一次状态变更，由流水线自动完成

见第二节前五页

注 时间安排与具体要求以指导书为准

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：选课平台的一次发布

第五讲 · 软件的生命周期

SJTU

场景设定：开放日前的一次发布

选课平台（思考题 1 阶段）

Nginx + 4 台 Tomcat + 1 台 MySQL（+ Redis 存储 session）

10/9 9:00 开放选课，峰值接近秒杀场景（L-04）

待上线变更：新的冲突检测规则按教学班判定时间冲突，需要在 enrollment 表新增 section_id 列，并修改选课接口

时间线：10/6 代码完成 → ？ 发布 → 10/9 9:00 开放

已知参数

一台 Tomcat 启动 + 预热约 1 min；优雅停止上限 30 s

蓝绿部署需额外开 4 台（按天计费）

功能开关：有配置中心时可热生效；仅有配置文件时需重启生效；两种情况均需讨论

引例 8:55 发现新规则存在缺陷，可以采取哪些措施?

课堂讨论

1 变更冻结

开放日前应提前几天冻结？冻结哪些内容（代码 / 配置 / 基础设施）？

冻结的代价是什么？例外流程如何规定？

2 schema 变更

本次加列如何实施才能保证可回滚？

应拆分为几步？各步在哪天发布？

3 8:55 发现「单双周」课程被误判为冲突

回滚（4 台滚动需要几分钟？蓝绿呢？）还是关闭功能开关？

哪种更快？各自的前提条件是什么？

4 回滚后如何处理 8:50 ~ 8:55 的请求？

(a) 合法选课被误拒

(b) 反之，冲突选课被误收

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

讨论要点

1 冻结

代码提前 2～3 天冻结，为 staging 预演和观察留出时间；配置变更允许，但须双人评审；基础设施冻结

例外情况走 hotfix 流程：最小改动 + 两人批准 + 回滚预案

冻结并非禁止修改，而是提高变更的准入门槛

2 expand–contract

10/6 发布①新增可空列（仅 schema）；10/7 发布②双写 + 回填，新规则置于开关之后（默认关闭）

10/8 在 staging 预演打开开关；10/9 之后再切读与收缩

任一步骤出现问题均可退回上一步，保留新列不影响运行

3 回退速度

开关（热生效）约为秒级，前提是 10/7 已加入开关，且 8:55 有具备权限并了解流程的人员执行关闭；若需重启生效，则退化为滚动

蓝绿约为秒级，前提是旧版本环境尚未撤除（多付一天 4 台费用）

滚动回滚约 4 ×（0.5 + 1）min ≈ 6 min；两台一批约 3 min，但峰值前容量减半；9:00 前勉强完成，风险较高

4 副作用

(a) 误拒：未产生写入，学生重试即可，但需发布公告

(b) 误收：冲突记录已写入数据库，回滚代码不会删除这些记录；需找出 8:50～8:55 按新规则写入的记录（写入时须带版本 / 规则标记）→ 逐条补偿（取消 + 通知，参见 L-03 Saga）→ 归还名额

结论 回退速度在发布前就已决定；代码可以回滚，副作用只能补偿；8:55 的选项取决于 10/6～10/8 的准备

本讲小结

- **软件是存在于时间轴上的对象**
  - 制品轴（跨版本）与实例轴（一次运行之内），部署是两轴的交点
- **生命周期中可变的只有两项**
  - 制品版本与配置版本：制品不可变、配置分离、环境一致
- **回滚有边界**
  - 发布、部署、放量三层的撤销速度依次递增；代码与配置可回退，数据只能向前演进
- **为回滚而设计**
  - schema 采用 expand–contract，功能采用开关，版本共存依靠向后兼容
- **实例生命周期短 → 无状态 → 支持滚动更新、扩缩容与自愈**

判断标准 该阶段的产出物是什么？能否回到上一个状态？（与「解决了什么问题、增加了什么成本」并列）

课后思考与参考资料

课后思考

1. 绘制小组项目的制品生命周期图（九个节点），标出已自动化和仍需人工完成的环节；目前能否直接回答「线上运行的是哪次提交」？

2. 选取一个使用过的软件、库或运行时的 EOL 公告，说明它对使用者的生命周期提出了哪些要求，以及迁移成本体现在何处。

3. 在选课平台中，哪些变更可以通过功能开关控制，哪些不能？（提示：开关作用于流量层，无法撤销 schema 变更和已写入的数据。）

参考资料

The Twelve-Factor App（12factor.net）：III Config、V Build-release-run、VI Processes、IX Disposability、X Dev/prod parity

D. Sato, ParallelChange, martinfowler.com (2014)；M. Fowler, BlueGreenDeployment (2010)、TolerantReader (2011)

P. Hodgson, Feature Toggles, martinfowler.com (2017)

J. Humble & D. Farley, Continuous Delivery (2010), Ch. 12

DORA, Accelerate State of DevOps Report（dora.dev）

Google SRE Book, Ch. 8 Release Engineering

Kubernetes docs: Liveness, Readiness and Startup Probes；Pod Lifecycle

endoflife.date

感谢聆听

饮 水 思 源　爱 国 荣 校